Heapsort solutions

06-E01 Heapify trace

Compare root 4 with children 9,8. Exchange with 9 to obtain [9,4,8,3,7,6,5]. At position 2, compare 4 with 3,7 and exchange with 7, obtaining [9,7,8,3,4,6,5]. Position 5 has no children, so stop. All parent-child inequalities now hold. Repair follows only the swapped subtree; the other subtree remains unaffected.

06-E02 Node heights and linear building

Height 0 has eight leaves, cost zero. Height 1 has four nodes, cost 4. Height 2 has two nodes, cost 4. Height 3 has one node, cost 3. The height sum is 11, bounded by a constant times 15.

Generally exactly floor(n/2ʰ) nodes have height at least h. Therefore $\sum_i height(i) = \sum_{h\geq 1} \lfloor n/2^h \rfloor < n$. Adding constant overhead per call still gives O(n). This accounts directly for an incomplete final level without discarding rounding terms.

06-E03 Priority update

Position 8's parent is 4, with key 7. Since 16>7, swap upward. Its new parent is position 2, with key 15; 16>15 requires another swap. Root 20 exceeds 16, so stop. The final array is [20,16,18,15,12,9,17,7]. The old key 2 has been replaced, not retained at the end. Each ascent moves the possible violation to one parent edge without changing other subtrees. A key decrease would instead require downward repair.

06-E04 An incorrect parent-index validator

Both children of root 10 satisfy 10≥1 and 10≥9, so the array is a legal heap. At c=3, the wrong rule uses ceil(3/2)=2 and tests 1≥9, falsely rejecting it. In 1-based indexing use floor(c/2) for every c=2..n. In 0-based indexing, a child j=1..n−1 has parent floor((j−1)/2), so test a[(j−1)//2]≥a[j]. The formulas cannot be mixed. A successful test on one large-root example would not prove a validator correct.

06-E05 Stability with equal keys

The initial heap is legal. Swap root with the last record to obtain (1,c),(2,b),(2,a). With heap size 2, heapify exchanges the first two, giving (2,b),(1,c),(2,a). The next root/end swap and size-1 heap yield (1,c),(2,b),(2,a). Keys are ordered, but a,b have reversed, so the sort is unstable. Checking key order and multiset alone cannot test stability; identities are needed. Adding original positions to the comparison key introduces another specification and storage cost.

Heapsort solutions (continued)

06-E06 Two repair directions after deletion

Deleting position 4 removes 10 and replaces it with 43. Since parent 20 at 2 is smaller, swap indices 4,2; root 50 stops ascent. The result is [50,43,45,20,15,40,44,5,9,12,14,35,39,42].

Independently delete root 50 and replace it with 43. Choose child 45, swap indices 1,3, then choose child 44 and swap 3,7. The result is [45,20,44,10,15,40,43,5,9,12,14,35,39,42].

If replacement exceeds its parent, the old inequalities parent≥deleted key≥children imply replacement exceeds the original children too, leaving only upward violations. Otherwise the parent edge is already legal and only downward heapify is needed. Each follows one path in O(log n). Locating the position is excluded; handle-based access additionally requires map updates on every swap.

Verification code appendix: heap_build_trace.py

```
"""Reproduce the ten-element linear-time heap construction trace."""


def heapify(a, i, size=None):
    size=len(a) if size is None else size
    while True:
        left = 2 * i + 1
        right = left + 1
        largest = i
        if left < size and a[left] > a[largest]:
            largest = left
        if right < size and a[right] > a[largest]:
            largest = right
        if largest == i:
            return
        a[i], a[largest] = a[largest], a[i]
        i = largest


def build_heap(a):
    a = list(a)
    stages = []
    for one_based in range(len(a) // 2, 0, -1):
        heapify(a, one_based - 1)
        stages.append((one_based, tuple(a)))
    return a, stages


def is_max_heap(a):
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Heapsort solutions (continued)

```
    return all(
        a[(child - 1) // 2] >= a[child]
        for child in range(1, len(a))
    )


def heap_sort_trace(values):
    a,_=build_heap(values)
    rows=[]
    for end in range(len(a)-1,0,-1):
        a[0],a[end]=a[end],a[0]
        heapify(a,0,end)
        assert is_max_heap(a[:end])
        assert a[end:]==sorted(a[end:])
        assert max(a[:end])<=a[end]
        rows.append((a[:end],a[end:]))
    assert a==sorted(values)
    return rows


def increase_key(a,position,key):
    i=position-1
    assert 0<=i<len(a) and key>=a[i]
    a[i]=key
    while i>0:
        parent=(i-1)//2
        if a[parent]>=a[i]:
            break
        a[parent],a[i]=a[i],a[parent]
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Heapsort solutions (continued)

```
        i=parent
    return a


def delete_at(a,position):
    i=position-1
    assert 0<=i<len(a)
    removed=a[i]
    last=a.pop()
    if i<len(a):
        a[i]=last
        if i>0 and a[i]>a[(i-1)//2]:
            increase_key(a,i+1,a[i])
        else:
            heapify(a,i)
    return removed


def run():
    source = [4, 1, 3, 2, 16, 9, 10, 14, 8, 7]
    expected = [16, 14, 10, 8, 7, 9, 3, 2, 4, 1]
    result, stages = build_heap(source)
    assert result == expected
    assert [i for i, _ in stages] == [5, 4, 3, 2, 1]
    assert stages[3][1] == (4, 16, 10, 14, 7, 9, 3, 2, 8, 1)
    assert is_max_heap(result)
    assert is_max_heap([10,1,9])
    assert not is_max_heap([10,11,9])
    sort_rows=heap_sort_trace(source)
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Heapsort solutions (continued)

```
    assert sort_rows[1]==([10,8,9,4,7,1,3,2],[14,16])
    updated=increase_key([20,15,18,7,12,9,17,2],8,16)
    assert updated==[20,16,18,15,12,9,17,7] and is_max_heap(updated)
    before=[50,20,45,10,15,40,44,5,9,12,14,35,39,42,43]
    up=before[:]
    assert delete_at(up,4)==10
    assert up==[50,43,45,20,15,40,44,5,9,12,14,35,39,42] and is_max_heap(up)
    down=before[:]
    assert delete_at(down,1)==50
    assert down==[45,20,44,10,15,40,43,5,9,12,14,35,39,42] and is_max_heap(down)
    from itertools import product
    cases=0
    for length in range(7):
        for values in product(range(3),repeat=length):
            built,_=build_heap(values)
            assert is_max_heap(built)
            assert sorted(built)==sorted(values)
            heap_sort_trace(values)
            cases+=1
    print(f"PASS heap_valid_odd_child_counterexample=True exhaustive_small_arrays={cases}
increase_key={updated} heapsort_rows={sort_rows}")
    print("PASS heap-build stages=5 arrays=5 final=" + str(result))
    return result


if __name__ == "__main__":
    run()
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Heapsort solutions (continued)

Verification code appendix: heap_operations.py

```
"""Build a max-heap and verify all parent-child relations."""

def max_heapify(a, i, size):
    while True:
        left, right = 2 * i + 1, 2 * i + 2
        largest = i
        if left < size and a[left] > a[largest]:
            largest = left
        if right < size and a[right] > a[largest]:
            largest = right
        if largest == i:
            return
        a[i], a[largest] = a[largest], a[i]
        i = largest


def build_heap(a):
    for i in range(len(a) // 2 - 1, -1, -1):
        max_heapify(a, i, len(a))


sample = [4, 1, 3, 2, 16, 9, 10, 14, 8, 7]
build_heap(sample)
assert all(sample[i] >= sample[2 * i + 1] for i in range(len(sample) // 2))
assert all(2 * i + 2 >= len(sample) or sample[i] >= sample[2 * i + 2] for i in
range(len(sample) // 2))
print(sample)
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study